

Reconfigurable and High-Efficiency Polynomial Multiplication Accelerator for CRYSTALS-Kyber

Minghao Li¹, Jing Tian¹, *Member, IEEE*, Xiao Hu¹, and Zhongfeng Wang¹, *Fellow, IEEE*

Abstract—Recently, the National Institute of Standards and Technology (NIST) has identified the first four quantum-resistant algorithms for post-quantum cryptography (PQC) standardization. CRYSTALS-Kyber (Kyber) is the only public-key encryption and key-establishment algorithm among them. In this article, we propose a reconfigurable, high-speed, and area-efficient polynomial multiplication accelerator for Kyber to facilitate its practical applications. The cornerstone of polynomial multiplication is the butterfly unit (BU) structure, composed of modular addition, subtraction, and multiplication. For the modular multiplication, we adopt the Barrett reduction method and reduce the size of operands leveraging the form of modulus with a novel formula transformation, which significantly decreases the computational complexity and increases the maximum clock frequency. On the hardware side, we make four BU modules constitute a binomial arithmetic core (Bi-Core) as the basic reconfigurable unit. The memory access scheme tailored for parallel processing is explored with data-reusing and memory-grouping methods, and a compact control logic is devised. The complete polynomial multiplication architecture is coded with Verilog and implemented on a Xilinx Artix-7 xc7a100t-3 device. Experiment results demonstrate that our implementations with different configurations all outperform the state-of-the-art works in area efficiency by up to 39% improvement in terms of area-time product (ATP). Moreover, the proposed design with four Bi-Cores achieves the fastest speed among existing designs.

Index Terms—Field-programmable gate array (FPGA), hardware, Kyber, lattice-based cryptography (LBC), modular multiplication, number theoretic transform (NTT), polynomial multiplication, post-quantum cryptography (PQC).

I. INTRODUCTION

THE POPULARLY used public-key cryptography algorithms, Rivest Shamir Adleman (RSA) [1] and elliptic curve cryptography (ECC) [2], are threatened by the development of powerful quantum computers [3], because their hard mathematical problems, the integer factoring and discrete logarithm problems, both can be efficiently solved by large-scale quantum computers with Shor's algorithm [4]. In order to

seek quantum-secure replacements, the National Institute of Standards and Technology (NIST) initiated the post-quantum cryptography (PQC) standardization process in 2016 [5]. After the third round of evaluation, in July 2022, a total of four candidate algorithms have been selected by NIST for standardization, and four additional algorithms will continue into the fourth round [6].

As one of the algorithms to be standardized, the key encapsulation mechanism (KEM) CRYSTALS-Kyber [7], also known as Kyber, is a kind of lattice-based cryptography (LBC), whose security is based on the hardness assumptions of solving the module learning with errors (Module-LWEs) problem [8]. The basic algebraic operation in module lattice is in the form of $\mathbf{A}\mathbf{s} + \mathbf{e}$, where $\mathbf{s}$ and $\mathbf{e}$ are k -dimension polynomial vectors and $\mathbf{A}$ is a $k \times k$ -dimension polynomial matrix with their elements in the polynomial ring $\mathbf{R}_q$. It should be noted that the security level of Kyber can be easily adjusted by changing the module dimension k . For example, three parameter sets Kyber512, Kyber768, and Kyber1024 are constructed with $k = 2, 3$, and 4 , which are equivalent to the NIST security levels of 1, 3, and 5, respectively. In the meanwhile, the parameter set for polynomial multiplication remains unchanged since the polynomial ring is fixed as $\mathbf{R}_q = \mathbb{Z}_{3329}[X]/(X^{256} + 1)$ in all of the security levels according to the latest version of Kyber.

In order to promote the practical application of Kyber, many researchers have focused on reducing the computational complexity and latency of the polynomial multiplication on hardware platforms. Huang et al. [9] presented the first pure hardware implementation of Kyber based on the field-programmable gate array (FPGA), where the bottom modules are reused to realize the related functions. Thereafter, many FPGA-based Kyber architectures [10], [11], [12], [13], [14], [15] have been proposed, where the polynomial multiplication is optimized with various techniques. Several recent works [16], [17], [18] make more dedicated improvements for the polynomial multiplication scheme of Kyber, exploring different tradeoffs between the throughput and area efficiency.

Number theoretic transform (NTT) is widely used for implementations of Kyber to optimize the polynomial multiplication, which has the advantage of quasi-linear complexity. The cornerstone for NTT/INTT hardware design is the butterfly unit (BU), which mainly consists of one modular multiplication, one modular addition, one modular subtraction, and two division-by-2 operations. The modular multiplication raises more attention than the others as it is much more complicated. Huang et al. [9] adopted the Montgomery reduction algorithm [19] to optimize the modular reduction. Chen et al. [10],

Manuscript received 7 July 2022; revised 9 October 2022; accepted 9 December 2022. Date of publication 19 December 2022; date of current version 19 July 2023. This work was supported in part by the National Natural Science Foundation of China under Grant 62104097; in part by the Key Research Plan of Jiangsu Province of China under Grant BE2022098; and in part by the High-Level Personnel Project of Jiangsu Province under Grant JSSCBS20210034. This article was recommended by Associate Editor N. Mentens. (Minghao Li and Jing Tian contributed equally to this work.) (Corresponding author: Zhongfeng Wang.)

The authors are with the School of Electronic Science and Engineering, Nanjing University, Nanjing 210023, China (e-mail: minghaoli@smail.nju.edu.cn; tianjing@nju.edu.cn; huxiao@smail.nju.edu.cn; zfwang@nju.edu.cn).

Digital Object Identifier 10.1109/TCAD.2022.3230359

Yao et al. [11], and Ma et al. [20] implemented the modular reduction by directly using the Barrett algorithm [21]. Zhang et al. [17] employed a set of look-up tables to implement the reduction unit, which is efficient when the prime q is small. Yarman et al. [18] proposed a new reduction algorithm by recursively utilizing $x \cdot 2^{\lceil \log q \rceil} \equiv x \cdot (2^{\lceil \log q \rceil} - q) \bmod q$. In this work, for BU implementation, we adopt the Barrett reduction algorithm and devise a compact modular multiplier based on a novel formula transformation by further leveraging the property of q to reduce the hardware complexity of BU. Meanwhile, the two division-by-2 operations of the BU are merged into the corresponding modular adder and subtractor.

Besides NTT and INTT, the other important operation of the polynomial multiplication is the coefficient-wise multiplication (CWM). In most of the LBC algorithms, such as NTRU [22] and CRYSTALS-Dilithium [23], CWM is defined as component-wise multiplication over $\mathbb{Z}_q$. However, it has to be pointed out that due to Kyber's special parameter set, CWM in Kyber is redefined as binomial multiplications over small polynomial rings such as $\mathbb{Z}_q[X]/(X^2 - \omega)$, which makes CWM much more complicated. For example, the latency required by CWM is $1.2 \times$ NTT in the polynomial multiplication accelerator proposed by [14]. Yarman et al. [18] divided CWM into five steps and reduced the latency of CWM to nearly $0.7 \times$ that of NTT at the price of a more complicated control logic. In this work, four BU modules are combined to form a basic reconfigurable unit, called binomial arithmetic core (Bi-Core), to serve as the processing elements in NTT, INTT, and CWM. For CWM, the Bi-Core is configured as a binomial multiplier to accelerate the CWM procedure, which achieves shorter latency than previous works.

The memory access scheme and control logic are essential for area-efficient polynomial multiplication implementation, and are difficult to be optimized for parallel processing especially considering the flexibility in the degree of parallelism [16], [24]. Many works [16], [17], [20] make use of a ping-pong memory to avoid memory access conflict in NTT, but this doubles the memory consumption. Moreover, the number of BUs is fixed and less than 4 in these works. Yarman et al. [18] proposed the first and the only flexible design for Kyber with an adjustable number of BUs based on Ahmert's memory access scheme [25]. Ahmert's scheme satisfies the bandwidth requirement of NTT without sacrificing the in-place advantage of the used NTT algorithm. However, in order to solve the high fan-out problem, the constant memory of twiddle factors is directly copied for each BU, which linearly increases the memory consumption. In this work, with the aim of parametric design with a flexible degree of parallelism, we extend Ahmert's scheme and compress the memory space by grouping twiddle factors. Ultracompact control logic is proposed to support this parallel memory access scheme, which appropriately schedules the data flow to Bi-Cores under NTT, INTT, and CWM configurations.

The main contributions of our work are summarized as follows.

- 1) A high-speed and low-complexity modular multiplier is proposed based on the Barrett reduction algorithm and a novel formula transformation.
- 2) A runtime reconfigurable processing element, called Bi-Core, is devised with two configurations: a) the butterfly unit for NTT/INTT and b) the binomial multiplier for CWM. Due to the hardware reconfigurability, our accelerator reaches a high resource utilization rate and achieves the best area efficiency in terms of area-time product (ATP) performance.
- 3) A parallel memory access scheme for polynomial multiplication of Kyber is presented by using the memory-grouping and data-reusing methods, which leads to an ultracompact control logic.
- 4) The degree of parallelism (i.e., the number of Bi-Cores) is parameterized in our Verilog design to fit different application scenarios with diverse throughput and area requirements. The hardware structure of control logic can be automatically generated using the parameterized Verilog program in the compile time.

The experiment results on a Xilinx Artix-7 FPGA device show that our accelerator with one single Bi-Core needs only $0.78 \mu s$ for NTT and $2.88 \mu s$ for one complete polynomial multiplication at the frequency of 303 MHz, achieving more than 39% ATP improvement compared with the prior arts. Moreover, the proposed design with four Bi-Cores achieves the fastest speed, whose ATP performance is still better than previous works.

The remainder of this article is organized as follows. In Section II, we introduce the notation used in this article, the Kyber protocol, and the algorithms of NTT and CWM. In Section III, we present the hardware architecture of our design, with the optimized Bi-Core and parallel memory access scheme. The FPGA implementation results are provided and compared with the state-of-the-art works in Section IV. Finally, we conclude this article in Section V.

II. BACKGROUND

A. Notation

The data width of the prime q is denoted by $W_q = \lceil \log q \rceil$. The quotient ring of integer polynomials is denoted by $\mathbf{R}_q = \mathbb{Z}_q[X]/(X^N + 1)$, in which the polynomials with coefficients in $\mathbb{Z}_q$ are reduced by $X^N + 1$. The maximum degree of any polynomial $f \in \mathbf{R}_q$ is $N - 1$, whose coefficients are in the range of $[0 : q)$. The i th coefficient of the polynomial f is denoted as f_i , i.e., $f = \sum_{i=0}^{N-1} f_i X^i$. For Kyber, the degree- $(N - 1)$ polynomial can be seen as a degree- $((N/2) - 1)$ polynomial of $Y = X^2$ written as follows:

$$f = (f_{N-1}X + f_{N-2}) \cdot Y^{\frac{N}{2}-1} + \dots + (f_3X + f_2) \cdot Y + (f_1X + f_0) \quad (1)$$

whose coefficients are degree-1 binomials. The i th binomial coefficient of f is noted as $f[i]$, i.e., $f[i] = f_{2i+1}X + f_{2i}$.

NTT of polynomial f is defined as $\tilde{f} \leftarrow \text{NTT}(f)$, where $\tilde{f}$ is the polynomial representation in NTT-domain, while the INTT is written as $f \leftarrow \text{INTT}(\tilde{f})$. The polynomial multiplication over $\mathbf{R}_q$ is defined by $h = f \cdot g \bmod (X^N + 1)$, and we abbreviate it as $h = f \cdot g$. The CWM between two polynomials performed in NTT-domain is denoted as $\tilde{h} = \tilde{f} \circ \tilde{g} = \text{CWM}(\tilde{f}, \tilde{g})$.

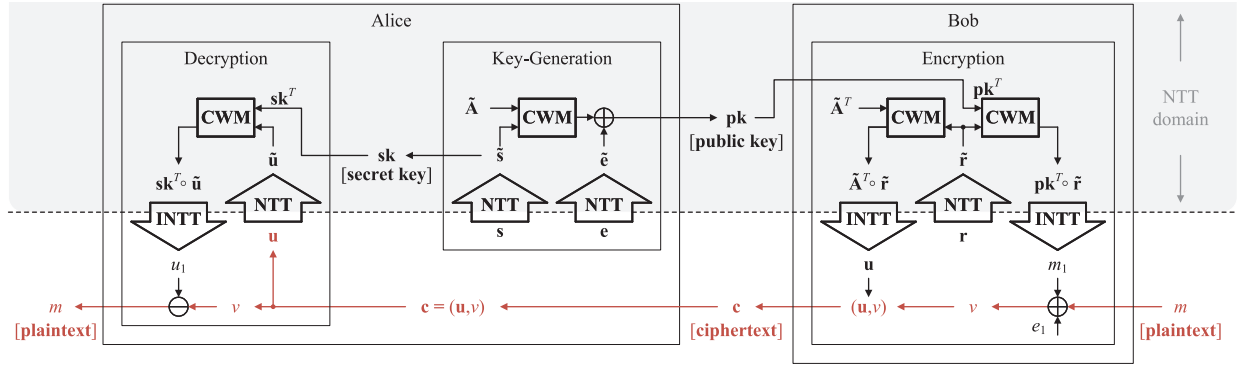


Fig. 1. Kyber's underlying public-key encryption scheme.

Besides, the binary number is marked by a subscript b . For example, the 1101_b equals to 13 in the decimal system. We use functions $\mathbf{inv}_k(\cdot)$ and $\mathbf{rev}_k(\cdot)$ to represent the bit-wise inverting and reversing operations for the lower k bits (k equals $\log N - 1$ if omitted). For instance, $\mathbf{inv}_3(1101_b) = 1010_b$ and $\mathbf{rev}_4(1101_b) = 1011_b$. If x is an integer but not a polynomial, $x[i]$ then represents the i th least significant bit of x , and $x[j : i]$ is used to denote the i th to j th bits of x . And the $\{\dots\}$ is used to represents bit splicing, e.g., $x[j : i] = \{x[j : k], x[k - 1 : i]\}$.

B. Kyber Protocol and Polynomial Multiplication

The underlying public key encryption scheme of Kyber is illustrated by Fig. 1, which is composed of three preliminaries, i.e., the *Key-Generation*, *Encryption*, and *Decryption*. As highlighted in red, through serials of matrix-vector and vector-vector multiplications in the NTT-domain, the message polynomial m is encrypted to $\mathbf{c}$ by Bob and then sent to Alice and decrypted. The polynomial multiplication is composed of three steps: NTT, INTT, and CWM. Taking the *Decryption* as an example, the multiplication between $\mathbf{u}$ and the secret-key $\mathbf{sk}$ is handled by: 1) transforming $\mathbf{u}$ to NTT-domain through NTT operation; 2) calculating CWM between $\mathbf{sk}^T$ and $\tilde{\mathbf{u}}$ in NTT-domain; and 3) turning back through INTT operation and obtaining the result polynomial u_1 . The frequent and computationally intensive polynomial multiplications form the bottleneck for the fast implementation of Kyber. For clarity, the NTT-based polynomial multiplication between polynomials p_1 and $\tilde{p}_2$, with $\tilde{p}_2$ in NTT-domain, is summarized as follows:

$$\text{PMult}(p_1, \tilde{p}_2) = \text{NTT}^{-1}(\text{CWM}(\text{NTT}(p_1), \tilde{p}_2)). \quad (2)$$

C. NTT and CWM

NTT is widely adopted in the implementations of LBC systems to speed up the polynomial multiplication over $\mathbf{R}_q$. Intuitively, NTT is mathematically similar to discrete Fourier transform (DFT), except that DFT is defined on the complex field $\mathbb{C}$ while NTT is defined on the integer ring $\mathbb{Z}_q$. Generally, in order to support N -point NTT-based multiplication, the input polynomial should be padded to $2N$ -point and an extra modulo reduction by $X^N + 1$ is required. However, this can be released by utilizing the negative wrapped convolution (NWC) technique [26]. In general, to support an N -point NWC-based NTT, the prime q is required to satisfy $q \equiv 1 \pmod{2N}$ such

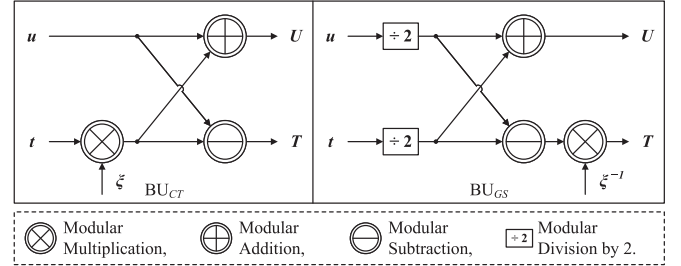


Fig. 2. Cooley-Tukey unit (BU_{CT}) and Gentleman-Sande unit (BU_{GS}).

that there is a primitive $2N$ th root of unity in $\mathbb{Z}_q$. However, Seiler et al. [27] proposed a variant NTT operation which is feasible for $q \equiv 1 \pmod{N}$, and the definition of CWM operation is also modified accordingly. Kyber adopts this algorithm and updates the parameters as $N = 256$ and $q = 3329$. The modified NTT and INTT are specified as follows:

$$\text{NTT} : \tilde{f}[i] = \sum_{j=0}^{\frac{N}{2}-1} f[j] \cdot \omega^{(2i+1)j} \pmod{q} \quad (3)$$

$$\text{INTT} : f[i] = \frac{2}{N} \sum_{j=0}^{\frac{N}{2}-1} \tilde{f}[j] \cdot \omega^{-i(2j+1)} \pmod{q} \quad (4)$$

where ω represents the N th primitive root of $\mathbb{Z}_q$. It is worth noting that $f[i] = f_{2i+1}X + f_{2i}$ is a degree-1 binomial. Similar to DFT, the NTT can be efficiently calculated by utilizing the butterfly structure. The in-place NTT algorithm based on Cooley-Tukey BU (BU_{CT}) [28] is given in Algorithm 1. In order to avoid the coefficient reversing operation, the Gentleman-Sande BU (BU_{GS}) is used [29] in INTT as shown in Algorithm 2. The butterfly structures used in Algorithms 1 and 2 are depicted in Fig. 2, which contains one modular multiplication, one addition, and one subtraction. Two extra division-by-2 operations are required by BU_{GS} . Compared with the school-book polynomial multiplication method [30], the multiplication complexity of NTT/INTT is reduced from $O(N^2)$ to $O(N \log N)$ by Algorithms 1 and 2.

CWM in Kyber is a multiplication of two binomials over the ring $\mathbb{Z}_{3329}[X]/(X^2 - \omega^{2i+1})$. For example, the $\tilde{h} = \text{CWM}(\tilde{f}, \tilde{g})$

Algorithm 1: Iterative in-Place NTT Algorithm Based on Cooley–Tukey Butterfly [28] for Kyber

Input: Input polynomial $f \in \mathbb{Z}_q[X]/(X^N + 1)$.

Input: Primitive root $\omega \in \mathbb{Z}_q$.

Output: $\tilde{f} \leftarrow NTT(f)$.

/* $(\log N - 1)$ stages in total */

```

1: for  $s = 0, 1, \dots, \log N - 2$  do
2:    $m \leftarrow 2^s, k \leftarrow N/4m$ .
3:   for  $i = 0, 1, \dots, m$  do
4:      $j_0 \leftarrow 2i \cdot k, j_1 \leftarrow (2i + 1) \cdot k - 1$ .
5:      $\xi \leftarrow \omega^{\text{rev}_{N/2}(m+i)}$ .
6:     for  $j = j_0, \dots, j_1$  do
7:        $u_1X + u_0 \leftarrow f[j], t_1X + t_0 \leftarrow f[j + k]$ .
8:        $(U_1, T_1) \leftarrow \text{BU}_{CT}(u_1, t_1, \xi)$ .
9:        $(U_0, T_0) \leftarrow \text{BU}_{CT}(u_0, t_0, \xi)$ .
10:       $f[j] \leftarrow U_1X + U_0, f[j + k] \leftarrow T_1X + T_0$ .
11:    end for
12:  end for
13: end for
14: return  $\tilde{f} \leftarrow f$ .
```

Algorithm 2: Iterative in-Place INTT Algorithm Based on Gentleman–Sande Butterfly [29]

Input: Input polynomial $\tilde{f} \in \mathbb{Z}_q[X]/(X^N + 1)$.

Input: Primitive root $\omega \in \mathbb{Z}_q$.

Output: $f \leftarrow NTT^{-1}(\tilde{f})$.

/* $(\log N - 1)$ stages in total */

```

1: for  $s = \log N - 2, \log N - 3, \dots, 0$  do
2:    $m \leftarrow 2^s, k \leftarrow N/4m$ .
3:   for  $i = 0, 1, \dots, m$  do
4:      $j_0 \leftarrow 2i \cdot k, j_1 \leftarrow (2i + 1) \cdot k - 1$ .
5:      $\xi^{-1} \leftarrow \omega^{-\text{rev}_{N/2}(m+i)}$ .
6:     for  $j = j_0, \dots, j_1$  do
7:        $U_1X + U_0 \leftarrow \tilde{f}[j], T_1X + T_0 \leftarrow \tilde{f}[j + k]$ .
8:        $(u_1, t_1) \leftarrow \text{BU}_{GS}(U_1, T_1, \xi^{-1})$ .
9:        $(u_0, t_0) \leftarrow \text{BU}_{GS}(U_0, T_0, \xi^{-1})$ .
10:       $\tilde{f}[j] \leftarrow u_1X + u_0, \tilde{f}[j + k] \leftarrow t_1X + t_0$ .
11:    end for
12:  end for
13: end for
14: return  $f \leftarrow \tilde{f}$ .
```

is computed by

$$\begin{aligned} \tilde{h}[i] &= \tilde{f}[i] \cdot \tilde{g}[i] = \left(\tilde{f}_{2i+1}X + \tilde{f}_{2i} \right) \cdot \left(\tilde{g}_{2i+1}X + \tilde{g}_{2i} \right) \\ &\quad \text{mod } X^2 - \omega^{\text{rev}_{N/2}(2i+1)} \end{aligned} \quad (5)$$

for $0 \leq i < N/2$. This is calculated with five modular multiplications and two modular additions in most previous works [11], [14], [18], [20], i.e.,

$$\begin{cases} \tilde{h}_{2i} = \tilde{f}_{2i} \cdot \tilde{g}_{2i} + \tilde{f}_{2i+1} \cdot \tilde{g}_{2i+1} \cdot \omega^{\text{rev}_{N/2}(2i+1)} \\ \tilde{h}_{2i+1} = \tilde{g}_{2i} \cdot \tilde{f}_{2i+1} + \tilde{f}_{2i} \cdot \tilde{g}_{2i+1} \end{cases} \quad (6)$$

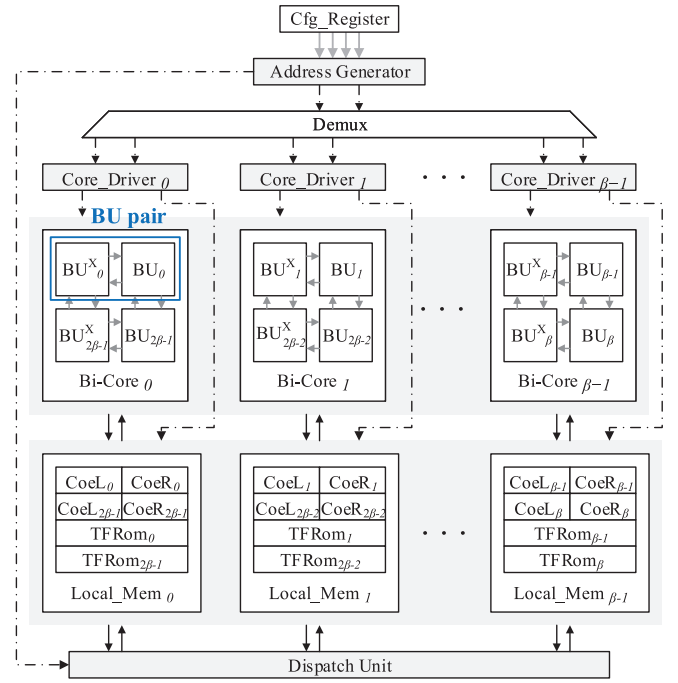


Fig. 3. Top-level architecture of the proposed polynomial multiplication accelerator.

The CWM between two N -point polynomials $\tilde{f}$ and $\tilde{g}$ is completed by using (6) for $N/2$ times, i.e., $5N/2$ modular multiplications in total. The multiplication complexity of CWM becomes significant because of the use of binomial form. Therefore, we will also take this step into consideration in terms of hardware optimization in the next section.

The constants derived from the primitive root ω are usually precomputed and stored on-chip for use, named as twiddle factors. The twiddle factors used for NTT, INTT, and CWM are, respectively, denoted by polynomials Ω , Ω' , and $\Omega^\dagger$, summarized as follows:

$$\begin{cases} \Omega = \omega^{\text{rev}(\frac{N}{2}-1)}X^{\frac{N}{2}-1} + \dots + \omega^{\text{rev}(1)}X + 1 \\ \Omega' = \omega^{-\text{rev}(\frac{N}{2}-1)}X^{\frac{N}{2}-1} + \dots + \omega^{-\text{rev}(1)}X + 1 \\ \Omega^\dagger = \omega^{\text{rev}(N-1)}X^{\frac{N}{2}-1} + \dots + \omega^{\text{rev}(3)}X + \omega^{\text{rev}(1)}. \end{cases} \quad (7)$$

III. POLYNOMIAL MULTIPLICATION ACCELERATOR

A. Top-Level Architecture

The top-level architecture of the polynomial multiplication accelerator is presented in Fig. 3.

The entire accelerator is designed to be reconfigurable, which can efficiently complete NTT, INTT, and CWM. The basic computing unit is the Bi-Core, which can be configured as four BUs for NTT/INTT or one binomial multiplier for CWM. On the top-level, there are $\beta = 2^\alpha$ Bi-Cores, where $0 \leq \alpha < \log N - 2$. Each Bi-Core only retrieves data from its local memory block (Local_Mem), and different Local_Mems communicate with each other through the Dispatch Unit. A pair of Bi-Core and Local_Mem is controlled by a Core_Driver. The address generator (AddrGen) is designed to parse the configuration information buffered in Cfg_Register and generate the reading addresses and other

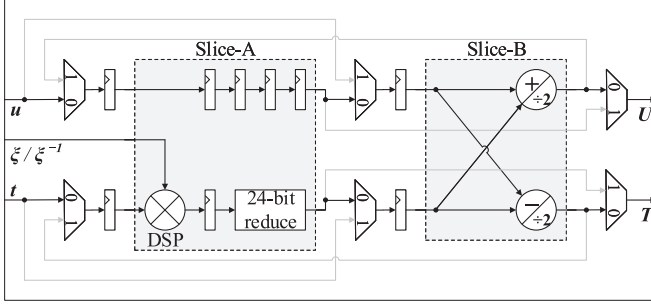
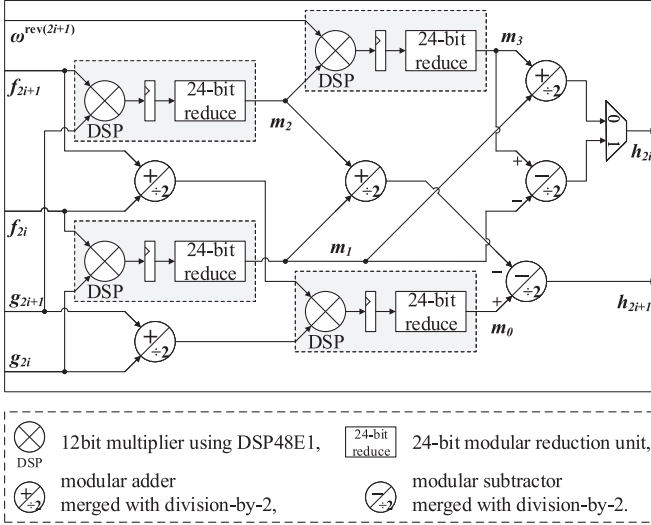
1st. BU_{CT} / BU_{GS} : Configuration for NTT / INTT2nd. Binomial Multiplier over $\mathbb{Z}_{3329} / (X^2 - \omega^{\text{rev}(2i+1)})$: Configuration for CWM

Fig. 4. Structure of Bi-Core.

control signals for NTT/INTT and CWM, which are passed to Core_Drivers through the demux to reduce the fan-out.

B. Bi-Core

The structure of Bi-Core is depicted by Fig. 4, including two kinds of configurations. For the 1st configuration, the Bi-Core forms four independent BUs, each of which is composed of one Slice-A and one Slice-B. BU_{CT} and BU_{GS} can be realized by controlling the sequence in which the data flows through the two slices using six multiplexers. The selection signals of the multiplexers are set to 0 when the BU is configured as BU_{CT} . Otherwise, the selection signals are set to 1 for BU_{GS} . The modular multiplication in Fig. 2 is implemented by Slice-A. The W_q -bit multiplication is realized by the DSP48E1 slice of FPGA, aiming for high speed. The $2W_q$ -bit output of DSP is then reduced within $\mathbb{Z}_{3329}$ by the reduction module, which is elaborately designed for short critical path and high area efficiency. To further reduce the hardware complexity, the two division-by-2 operations in BU_{GS} are merged into the adjacent modular addition and subtraction. The modified modular adder/subtractor performs $a \pm b \bmod q$ in NTT, and performs $2^{-1} \cdot (a \pm b) \bmod q$ in INTT.

For the 2nd configuration, as shown in the lower part in Fig. 4, the submodules of four BUs are appropriately reorganized to form a binomial multiplier over $\mathbb{Z}_q / (X^2 - \omega^{\text{rev}(2i+1)})$,

which is used to compute (5). We reformulate (6) as follows:

$$\begin{cases} m_0 = (\tilde{f}_{2i} + \tilde{f}_{2i+1})(\tilde{g}_{2i} + \tilde{g}_{2i+1}) \\ m_1 = \tilde{f}_{2i} \cdot \tilde{g}_{2i} \\ m_2 = \tilde{f}_{2i+1} \cdot \tilde{g}_{2i+1} \\ m_3 = m_2 \cdot \omega^{\text{rev}_{N/2}(2i+1)} \\ \tilde{h}_{2i} = m_1 + m_3 \\ \tilde{h}_{2i+1} = m_0 - (m_1 + m_2) \end{cases} \quad (8)$$

which reduce the number of multiplications to 4, i.e., m_0, m_1, m_2 , and m_3 . In this way, the four Slice-As in Bi-Core could be fully utilized in all of NTT, INTT, and CWM schemes.

1) *24-Bit Reduction Module Based on the Proposed Low-Complexity Barrett Reduction Algorithm for Kyber*: The reduction module in Slice-A is elaborately devised based on the Barrett reduction algorithm [21] and a novel formula transformation. Since the NTT-applicable primes satisfy that $q \equiv 1 \bmod N$, they can be written in the form of $q = \delta \cdot 2^e + 1$, where δ is a positive integer and $e = \log N$. Leveraging this property, the $2W_q$ -bit product $c \in [0, 2^{2W_q})$ from the DSP unit is correspondingly decomposed as $c = c_1 \cdot 2^e + c_0$, where $c_0 = c[e-1 : 0]$ and $c_1 = c[2W_q-1 : e]$. Inspired by the fast reduction algorithm proposed by [31], we replace the large modulus q with the small modular δ by a formula transformation

$$\begin{aligned} c_1 \cdot 2^e + c_0 \bmod q & \equiv \left((c_1 \cdot 2^e \bmod (q-1)) - \left\lfloor \frac{c_1 \cdot 2^e}{q-1} \right\rfloor + c_0 \right) \bmod q \\ & \equiv \left((c_1 \cdot 2^e \bmod (\delta \cdot 2^e)) - \left\lfloor \frac{c_1 \cdot 2^e}{\delta \cdot 2^e} \right\rfloor + c_0 \right) \bmod q \\ & = ((c_1 \bmod \delta) \cdot 2^e - \lfloor c_1 / \delta \rfloor + c_0) \bmod q \\ & = r_1 \cdot 2^e - Q_1 + c_0 \bmod q \end{aligned} \quad (9)$$

where $Q_1 = \lfloor c_1 / \delta \rfloor$ and $r_1 = c_1 \bmod \delta$.

According to the original Barrett reduction algorithm [21], Q_1 and r_1 in (9) can be, respectively, estimated by Q'_1 and r'_1

$$\begin{cases} Q'_1 = \left\lfloor \frac{c_1 \cdot \zeta}{2^{2W_q-e}} \right\rfloor, \quad \zeta := \lfloor 2^{2W_q-e} / \delta \rfloor \\ r'_1 = c_1 - Q'_1 \cdot \delta \end{cases} \quad (10)$$

where the estimation deviation of Q_1 is less than 1, i.e., $0 \leq Q_1 - Q'_1 \leq 1$. However, considering $(\delta, e, W_q) = (13, 8, 12)$ for $q = 3329$ in Kyber, more aggressive bit-level improvements to (10) are presented in this article.

For the precomputed constant value $\zeta = \lfloor 2^{16}/13 \rfloor = 1001110110000_b$, we use the canonical signed digit (CSD) arithmetic [32] to simplify the multiplication with ζ , where ζ is written as $101000\bar{1}0\bar{1}0000_b$ and $\bar{1}$ denotes -1 . By reusing the component of 101_b , ζ can be further simplified as $101_b \cdot 100000\bar{1}_b \cdot 10000_b = (2^2 + 1)(2^6 - 1)2^4$. Based on this observation, the calculation of Q'_1 in (10) can be decomposed into three steps

$$\begin{cases} \text{step 1. 18-bit addition:} & d_{-1} = c_1 \cdot 2^2 + c_1 \\ \text{step 2. 25-bit subtraction:} & d_{-2} = d_{-1} \cdot 2^6 - d_{-1} \\ \text{step 3. 12-bit truncation:} & Q'_1 = \lfloor d_{-2} \cdot 2^{-12} \rfloor. \end{cases} \quad (11)$$

In [33], it is proven that the complexity of the multiplication with ζ in the Barrett algorithm can be reduced by truncating

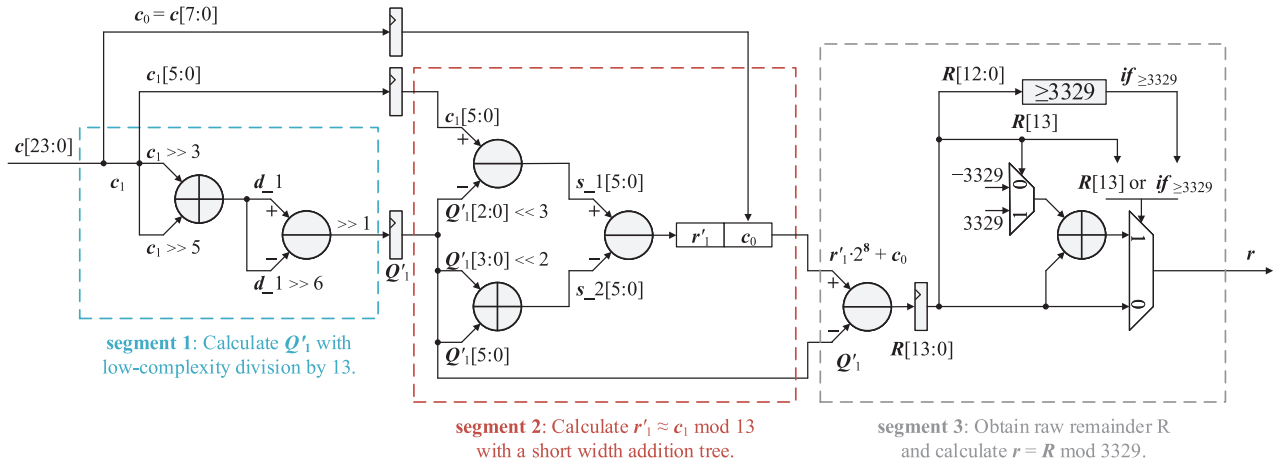


Fig. 5. 24-bit reduction module.

the lowest ρ bits of c_1 at the price of slightly increasing the precision of ζ to σ bits. To guarantee that $0 \leq Q_1 - Q'_1 \leq 1$, it is required that $\rho \leq \lceil \log \delta \rceil - 2$ and $\sigma \geq 2W_q - e + 1$. However, there is still a margin for more aggressive bit width reduction. Combined with (11), the width of c_1 is gradually reduced in two steps in this work

$$\begin{cases} \text{step 1: } d_1 = \lfloor c_1 \cdot 2^{-\rho_1} \rfloor + \lfloor c_1 \cdot 2^{-(\rho_1+2)} \rfloor \\ \text{step 2: } d_2 = \lfloor d_1 \cdot 2^{-\rho_2} \rfloor - \lfloor d_1 \cdot 2^{-(\rho_2+6)} \rfloor \\ \text{step 3: } Q'_1 = \lfloor d_2 / 2^{4-(\rho_1+\rho_2)} \rfloor \end{cases} \quad (12)$$

where the parameters ρ_1 and ρ_2 are introduced to indicate the widths of data truncation. Larger ρ_1 and ρ_2 mean shorter data widths and higher area efficiency of the adder and subtractor, but lead to a larger deviation from Q_1 , i.e., larger $Q_1 - Q'_1$. Adjusting ρ_1 and ρ_2 make a tradeoff between the complexity of estimation of Q_1 and the complexity of the following operations.

As for r'_1 in (10), in view of the limited data width of r'_1 , we extend the bit truncation method proposed in [33], and modify the calculation of r'_1 as follows:

$$\begin{aligned} r'_1 &= c_1 - Q'_1 \cdot 13 \\ &= c_1 - (Q'_1 \cdot 2^3 + Q'_1 \cdot 2^2 + Q'_1) \\ &= c_1[\sigma - 1 : 0] - Q'_1[\sigma - 4 : 0] \cdot 2^3 \\ &\quad - Q'_1[\sigma - 3 : 0] \cdot 2^2 - Q'_1[\sigma - 1 : 0] \end{aligned} \quad (13)$$

where $\sigma = \lceil \log r'_1 \rceil \geq \lceil \log \delta' \rceil$ denotes the width of r'_1 . Larger (ρ_1, ρ_2) , i.e., larger $Q_1 - Q'_1$, results in larger σ and thus, increases the computational complexity for r'_1 .

Based on the formula transformation in (9) and the simplified calculations for Q'_1 and r'_1 in (12) and (13), the low-complexity Barrett reduction algorithm for Kyber is summarized in Algorithm 3. Here, the function MOD in step 9 reduces the raw remainder R to the range of $[0, q)$ by adding R with $k \cdot 3329$, where $k \in \{-2, -1, 0, 1, 2\}$. The deviation between the raw remainder R and the correct residue r varies with (ρ_1, ρ_2) , which affects the implementing complexity of the function MOD .

Adjusting the truncation parameters (ρ_1, ρ_2) make a tradeoff between the computational complexity of segment 1 and

TABLE I
BRUTE-FORCE SEARCHING RESULTS FOR (ρ_1, ρ_2, σ)

ρ_1	ρ_2	σ	Range of R	# Slice ¹
-2	-6	4	$(-2q, 2q)^2$	64
-1	-3	5	$(-2q, q)$	56
1	-1	6	$(-q, 2q)$	48
3	0	6	$(-q, 2q)$	40

¹ The number of Slices utilized by the hardware implementation of Alg. 3.

² $q = 3329$.

Algorithm 3: Low-Complexity Barrett Reduction for Kyber

Input: $c = a \cdot b \in [0, 2^{24})$,
the truncation parameters ρ_1, ρ_2 , and σ .
Output: $r \equiv c \bmod 3329$.

- 1: $c_1 \leftarrow c[23 : 8], c_0 \leftarrow c[7 : 0]$.
- segment 1. Calculate $Q'_1 \approx \lfloor c_1/13 \rfloor$:** $\triangleright$ Eq. (12)
- 2: $d_1 \leftarrow \lfloor c_1 \cdot 2^{-\rho_1} \rfloor + \lfloor c_1 \cdot 2^{-(\rho_1+2)} \rfloor$.
- 3: $d_2 \leftarrow \lfloor d_1 \cdot 2^{-\rho_2} \rfloor - \lfloor d_1 \cdot 2^{-(\rho_2+6)} \rfloor$.
- 4: $Q'_1 \leftarrow \lfloor d_2 / 2^{4-(\rho_1+\rho_2)} \rfloor$.
- segment 2. Calculate $r'_1 \approx c_1 \bmod 13$:** $\triangleright$ Eq. (13)
- 5: $s_1 \leftarrow c_1[\sigma - 1 : 0] - Q'_1[\sigma - 4 : 0] \cdot 2^3$.
- 6: $s_2 \leftarrow Q'_1[\sigma - 3 : 0] \cdot 2^2 + Q'_1[\sigma - 1 : 0]$.
- 7: $r'_1 \leftarrow s_1[\sigma - 1 : 0] - s_2[\sigma - 1 : 0]$.
- segment 3. Obtain the raw remainder R and reduce it to $\mathbb{Z}_{3329}$:**
- 8: $R \leftarrow \{r'_1, c_0\} - Q'_1$. $\triangleright$ Eq. (9)
- 9: $r = \text{MOD}(R, 3329)$.
- 10: **return** $r \in \mathbb{Z}_{3329}$.

that of segments 2 and 3. To minimize the hardware complexity, a brute-forth searching for (ρ_1, ρ_2) is conducted to choose the optimized parameters. Parts of the search results are shown in Table I, where the dependent variables includes σ , the range of R , and the resource consumption of implementing Algorithm 3 on FPGA. We choose the optimized parameters $(\rho_1, \rho_2) = (3, 0)$, and the corresponding low-complexity hardware implementation is illustrated by Fig. 5, where the

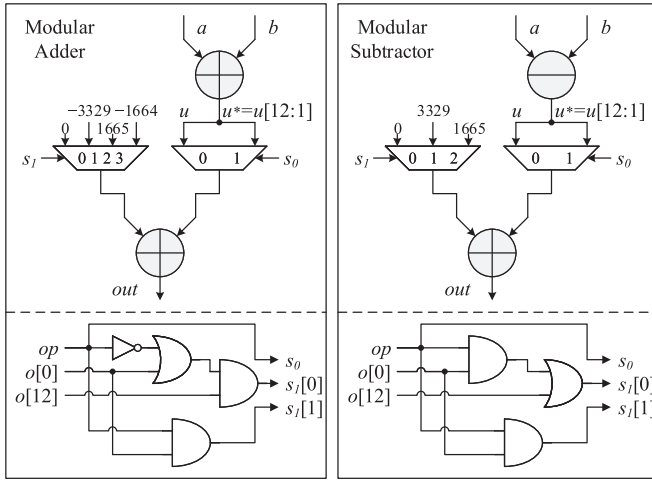


Fig. 6. Modified modular adder and subtractor merged with divisions.

two-stage pipeline design contributes to a very short critical path.

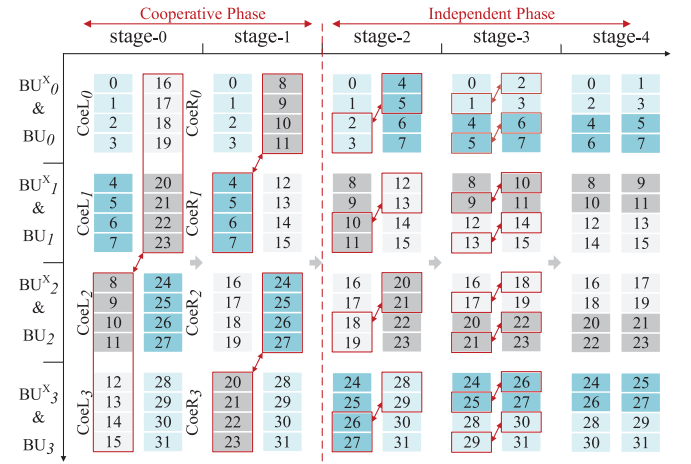
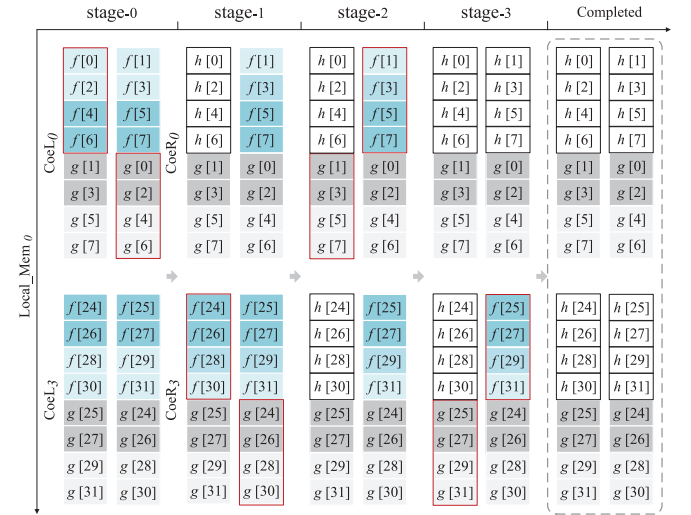
2) *Modified Modular Adder and Subtractor*: The hardware designs of the modified modular adder and subtractor are illustrated by Fig. 6, whose inputs and output are all 12-bit integers (sometimes greater than 3329). The division-by-2 operation is integrated into the modular addition and subtraction with insignificant area increase. In the following, the modular adder is described in detail, while the modular subtractor is omitted because of a similar design methodology.

When modular adder is configured for NTT/CWM operation ($op = 0$), it performs $(a + b) \bmod 3329$. The 13-bit sum u will be subtracted by 3329, if u exceeds 12-bit (i.e., $u[12] = 1$). When modular adder is configured for INTT ($op = 1$), it performs $2^{-1} \cdot (a + b) \bmod 3329$. Dividing u by 2 over $\mathbb{Z}_{3329}$ can be simply performed using one addition, because $2^{-1} \cdot u = u[12 : 1] + 2^{-1} \cdot u[0] = u^* + 1665 \cdot u[0]$, where $u^* = u[12 : 1]$. Therefore, the u^* is passed to the second adder and added with 1665 according to whether $u[0] = 1$. However, if $u[12] = 1$, the $u^* + 1665$ may exceed 12 bits. In this case (i.e., $u[0] = 1$ and $u[12] = 1$), the u^* is added with $1665 - 3329 = -1664$. It can be proven that $u^* - 1664$ is no more than 12 bits, because, as shown by Fig. 4, at least one of a and b is from the 24-bit reduction module and, thus, less than 3329. The control signals s_0 and s_1 of the two multiplexers are generated with several logic gates, and the critical path of the modular adder/subtractor is approximately two 12-bit additions.

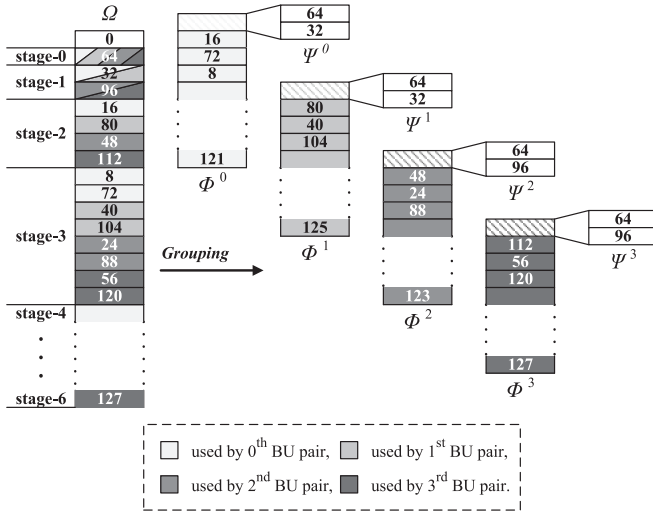
C. Parallel Memory Access Scheme and Control Logic

A significant challenge for polynomial multiplication hardware design is managing the complex memory access schedule with compact control logic [25], [34]. In this work, the memories for polynomials and twiddle factors are first grouped as many Local_Mems to support parallel memory accessing. Then the twiddle factors are fully reused to reduce BRAM consumption. Finally, a compact control logic is developed based on these optimizing methods.

1) *Data Flow Schedule*: The proposed design implements NTT using the in-place iterative scheme shown in Algorithm 1,

Fig. 7. Data flow example of NTT ($N = 64$ and $\beta = 2$).Fig. 8. Data flow example of CWM ($N = 64$ and $\beta = 2$).

which consists of $\log N - 1$ stages and $N/2$ butterfly operations at each stage. The memory access is scheduled based on the flexible shuffling scheme proposed by Mert et al. [34]. Fig. 7 gives a data flow example for a 64-point NTT with two Bi-Cores, where each block labeled by the number i represents the binomial $f[i]$. The binomial $f[i] = f_{2i+1}X + f_{2i}$ is stored as a 24-bit value $\{f_{2i+1}, f_{2i}\}$. The coefficients of polynomial f are divided into 2β equal parts, and each piece is stored in the left/right coefficient RAM, named Coel/Coer in the Local_Mems, as shown in Fig. 3. The coefficient RAMs are implemented as simple-dual-port RAMs with RAMB18 slices on the FPGA device. For each clock cycle, two binomials, i.e., the $f[j]$ and $f[j + k]$ at step 7 of Algorithm 1, are read from the same address of Coel and Coer. Then, they are sent to a pair of BU^X and BU in the corresponding Bi-Core to perform steps 8 and 9 of Algorithm 1. In preparing for the next stage, the outputs of BUs are transferred to different Coels and Coers, as noted by the red block in Fig. 7. This data exchange procedure is handled by the Dispatch Unit shown by Fig. 3. The INTT performs the inverse process of NTT similarly, which is very similar to NTT and omitted here.

Fig. 9. Example of grouping twiddle factors by $\beta = 2$.

Instead of initiating two separate sets of memories for polynomials f and g like most previous works, we adopt a cross-storage strategy as illustrated by Fig. 8. For example, at the first clock cycle in stage 0, the binomials $\tilde{f}[0]$ and $\tilde{g}[0]$ are, respectively, read from CoeL and CoeR of Local_Mem₀. Then, their multiplication result $\tilde{h}[0]$ from Bi-Core₀ is written back to CoeL overwriting $f[0]$. In this way, the two polynomials could share the same storage block and control logic without sacrificing the throughput.

2) *Twiddle Factors Grouping and Reusing*: Instead of copying the twiddle factors for each BU as [18], the twiddle factors in Ω are properly grouped to facilitate the parallel processing. Fig. 9 gives an example of twiddle factors grouping for NTT with $\beta = 2$ Bi-Cores (i.e., $2\beta = 4$ BU pairs).

Here, the numbers in blocks represent the exponents of the primitive roots in Ω . The stage numbers on the left indicate which twiddle factors are used at the corresponding NTT stage. For each stage, the twiddle factors used by different BU pairs are divided into 2β parts and stored in Φ pieces. Since the number of the twiddle factors used in stage 0 to stage α is less than 2β , these twiddle factors are copied, divided, and stored in Ψ pieces. The twiddle factors in Ψ^τ share the zero address of Φ^τ . In conclusion, the twiddle factors are split into equal parts for BU pairs, i.e., $\Omega \rightarrow \{(\Psi^\tau, \Phi^\tau) \mid \tau = 0, 1, \dots, 2\beta - 1\}$. A pair of (Ψ^τ, Φ^τ) is stored in TFRom _{τ} of the Local_Mem in Fig. 3, which is implemented by distributed ROMs. During the run time, each Bi-Core only retrieves the twiddle factors from its Local_Mem.

The sizes of Ψ^τ and Φ^τ are, respectively, $W_q(\alpha + 1)$ and $W_q \cdot N/4\beta$ bits, and the total memory size is $W_q(2\beta(\alpha + 1) + N/2) = W_q(2^{\alpha+1}(\alpha + 1) + N/2)$. Compared with copying twiddle factors for every BU as [18], which needs memory in total size of $W_q \cdot N/2 \cdot 2^{\alpha+1}$ bits, the grouping method achieves a compression ratio of $([2(\alpha + 1)]/N) + (1/2^{\alpha+1})$. The compression ratio decreases as the degree of parallelism increases, and reaches its minimum ($2\log N/N$) when NTT works in fully parallel ($\alpha = \log N - 2$).

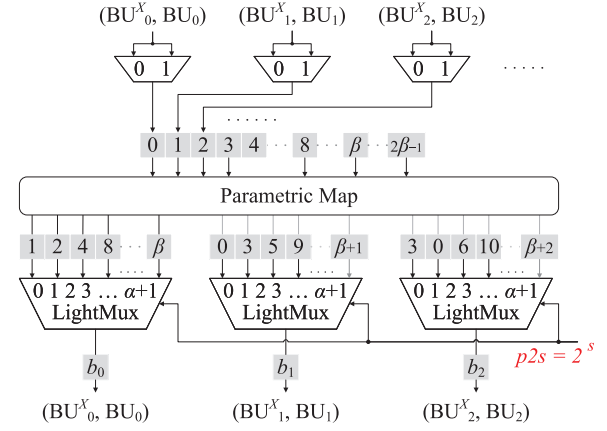


Fig. 10. Data exchange module, Dispatch Unit.

Considering the symmetry property of generation group, the Ω can be reused to generate Ω' and $\Omega^\dagger$. Given $\omega^{N/2} \equiv -1 \pmod{q}$, we have $\Omega'_\chi = \omega^{-\text{rev}(\chi)} = -\omega^{(N/2) - \text{rev}(\chi)}$. It can be proven that $(N/2) - \text{rev}(\chi) = \text{rev}(\text{inv}_{k(\chi)}(\chi))$, where $k(\chi) = \lfloor \log \chi \rfloor$. Therefore, it holds that $\Omega'_\chi = -\Omega_{\text{inv}_{k(\chi)}(\chi)}$. Combined with the twiddle factor grouping method, the reusing scheme for INTT is

$$\begin{cases} \Psi^\tau_\chi = -\Psi^{2\beta-1-\tau}_\chi, & (0 \leq \chi < \alpha) \\ \Phi^\tau_\chi = -\Phi^{2\beta-1-\tau}_{\text{inv}_{k(\chi)}(\chi)}, & (0 \leq \chi < \frac{N}{2\beta}) \end{cases} \quad (14)$$

for $0 \leq \tau < 2\beta$. Here, the (Ψ^τ, Φ^τ) is derived by grouping Ω' according to Fig. 9. For the convenience of twiddle factors reusing, the TFRom _{τ} of (Ψ^τ, Φ^τ) and TFRom _{$2\beta-1-\tau$} of $(\Psi^{2\beta-1-\tau}, \Phi^{2\beta-1-\tau})$ are placed in the same Local_Mem _{τ} as shown by Fig. 3. Therefore, in INTT, the twiddle factors are reused by: 1) replacing the reading addresses χ of TFRoms by its inverse, i.e., $\text{inv}_{k(\chi)}(\chi)$ and 2) exchanging the two twiddle factors read from one Local_Mem. The extra negative sign is merged into the subtraction operation of BU by swapping inputs, which is omitted in Fig. 4.

We extend this method to generate $\Omega^\dagger$, and the obtained twiddle factors reusing scheme for CWM is

$$\Omega^\dagger_j = \begin{cases} \Phi^\tau_{N/2\beta+\chi}, & \chi \bmod 2 = 0 \\ -\Phi^\tau_{N/2\beta+\chi}, & \chi \bmod 2 = 1 \end{cases} \quad (15)$$

where the $\Omega^\dagger_\tau$ is defined as the τ th block from dividing $\Omega^\dagger$ into 2β equal parts. A further observation to the storage pattern in Fig. 7 shows that, in the last stage, all $\tilde{f}[\chi]$ with even indexes (i.e., $\chi \bmod 2 = 0$) are stored in CoeL, and those with odd indexes (i.e., $\chi \bmod 2 = 1$) are stored in CoeR. Therefore, in CWM, the twiddle factors are reused by: 1) replacing the reading address of TFRoms by $(N/2\beta) + \chi$ and 2) using the addition result as the output h_{2i} when processing the binomials in CoeL, and using the subtraction result when processing those in CoeR, as shown by the 2nd configuration of Fig. 4.

3) *Compact Control Logic*: We devise the control logic based on the parallel memory access scheme introduced above. The hardware structure of the control logic is defined by parameters β and α and can be automatically generated during the compile time.

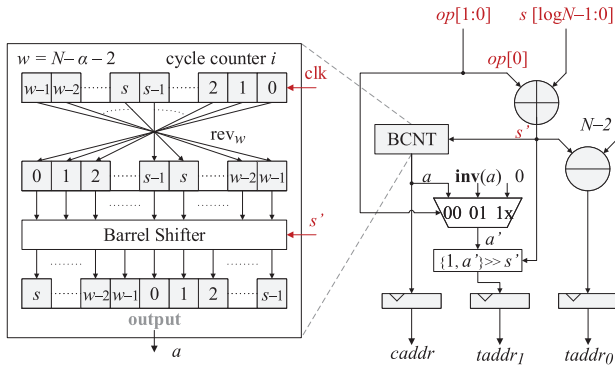


Fig. 11. AddrGen.

According to whether there is data exchange between different Local_Mems, the data flow of NTT is divided into two processing phases: 1) the cooperative phase and 2) the independent phase, as noted in Fig. 7. In the cooperative phase (the first $\alpha + 1$ stages, e.g., stages 0–1 in Fig. 7), the binomials are transferred among CoeLs and CoeRs that belong to different BU pairs. This procedure is properly managed by a data exchange module named Dispatch Unit, which is illustrated by Fig. 10. First, a total of 2β result binomials (array x) from all BU pairs are sent to the Dispatch Unit. Then, a parametric routing module named Parametric Map sorts out $\alpha + 1$ candidate binomials (array y_i) for each BU pair (BU_i^X , BU_i) based on the observation that

$$y_i[k] = x[\text{bitwise_XOR}(i, k)] \quad (16)$$

for $k = 0, 1, \dots, \alpha + 1$ and $i = 0, 1, \dots, 2\beta$. Finally, according to specific NTT stage, $b_i = y_i[s]$ is selected and sent to (BU_i^X , BU_i) by a light-weight multiplexer named LightMux, where s is the stage identifier as defined in Algorithm 1.

In the independent phase (e.g., stages 2–5 in Fig. 7), the BU pairs work independently, and no data is exchanged among them. However, the dynamic memory access pattern is complicated. For example, in the first cycle of stage 2, the binomials $f[0]$ and $f[4]$ are read from CoeL and CoeR in Local_Mem₀ and then sent to (BU_0^X , BU_0) to be processed. However, the binomials $f[2]$ and $f[6]$ must be read at the next cycle. Otherwise, the output binomial $f[4]$ from Bi-Core₀ would overwrite $f[2]$ and cause a read–write conflict. Different from [18] and [25], we devised an innovative AddrGen to handle this dynamic access pattern, which is illustrated by Fig. 11. The reading address $caddr$ for CoeLs and CoeRs is generated by a barrel counter (BCNT). By shifting the least significant bit of a cycle counter according to different stages, the output address sequence of BCNT can adequately eliminate the read–write conflicts. For instance, the sequence of $caddr$ is (0, 2, 1, 3) at stage 2 and (0, 1, 2, 3) at stage 3 for the example of Fig. 7. The other outputs ($taddr_0$, $taddr_1$) serve as the reading addresses for the (Ψ^r , Φ^r) in TFRoms, which are derived from $caddr$. The $\text{inv}_{k(\chi)}$ operation in (14) is taken by: 1) bit-wise inverting $caddr$ [i.e., $a' = \text{inv}(a)$] and 2) right shifting $\{1, a'\}$ by s' bits. By configuring the 2-bit value op , the AddrGen can be reused in NTT ($op = 00_b$), INTT ($op = 01_b$), and CWM ($op = 10_b/11_b$).

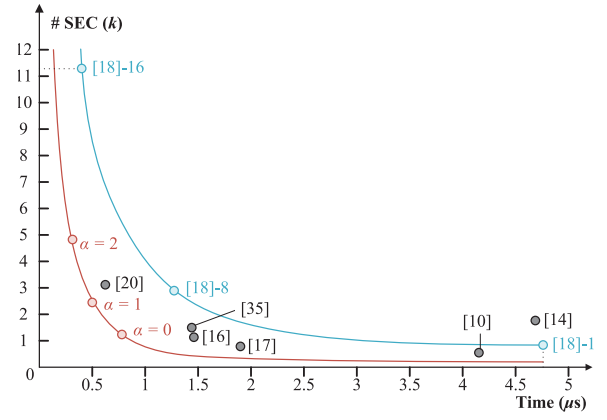


Fig. 12. Correlation between area and time.

IV. IMPLEMENTATION RESULTS AND COMPARISON

For a fair comparison, we implement our design on a Xilinx Artix-7 FPGA platform, which is popular for Kyber prototyping [13]. The Xilinx Vivado 2018.3 EDA platform is adopted. We experiment three hardware designs with one ($\alpha = 0$), two ($\alpha = 1$), and four ($\alpha = 2$) Bi-Cores to explore different tradeoffs between speed and area efficiency. In order to take the DSPs and BRAMs into consideration, we use the slice equivalent cost (SEC) as the metric for area evaluation. The area is calculated by $\#SEC = \#BRAMs \times 200 + \#DSPs \times 100 + \#Slices$, following the same approximation methods used in [30], i.e., one DSP block and one 36-kB BRAM equal to 102.4 and 196.2 slices, respectively. The area efficiency is evaluated by ATP, which is computed as $\#SEC \times \text{Time}$.

We compare the NTT performance of our designs with those in [10], [14], [16], [17], [18], [20], and [35] in Table II. Due to the fully optimized 24-bit reduction module, the proposed three designs can be run at maximum frequencies of 303, 282, and 273 MHz, respectively. Benefiting from the efficient parallel memory access scheme, only 235, 140, and 84 clock cycles are required for the three implementations to complete an NTT/INTT, respectively, which include 5, 9, and 12 cycles for function reconfiguration. It can be seen that our designs all achieve better area efficiency than existing works. Especially, the light-weight design ($\alpha = 0$) obtains 39% ATP improvement over the state-of-the-art [16]. The design with $\alpha = 2$ is devised for high throughput and low latency. It is seen that this design with four Bi-Cores achieves the fastest speed, costing only 0.31 μs to complete one NTT/INTT, whose ATP becomes worse but is still better than previous works. The design with two Bi-Cores ($\alpha = 1$) serves as a balanced solution, which ranks the third fastest and achieves 24% ATP improvement.

In order to clearly demonstrate the area-time tradeoff of the related works, the correlation of Area versus Time is depicted by Fig. 12. Our results are marked by red dots, those of [18] are denoted by blue dots, and the others are marked in gray. Our results and those of [18] are both fitted using inverse proportional functions as shown by the red and blue curves. Compared to [18] which also aims for parametric design with

TABLE II
NTT IMPLEMENTATION RESULTS AND COMPARISON TO PRIOR WORKS

Work ¹	# LUTs	# FFs	# DSPs	# BRAMs ²	# Slices ³	Frequency (MHz)	Latency (cc)	Time (μs)	ATP (# SEC ⁴ $\times \mu s$)
[10]	533	514	1	1.5	198 *	246	1030	4.18	2,499
[17]	609	640	2	2	232 *	257	490	1.90	1,580
[14]	360	145	3	2	187	115	940	9.32	8,267
[14]	737	290	6	4	371	115	474	4.68	8,288
[16]	801	717	4	2	290 *	222	324	1.46	1,591
[35]	1,549	788	4	2 ⁵	635	159	228	1.43	2,052
[20]	5,181	4,833	16	0	1,468	227	143	0.63	1,933
[18]-1	948	352	1	2.5	281 *	190	904	4.76	4,194
[18]-4	2,543	792	4	9	735 *	182	232	1.27	3,727
[18]-16	9,508	2,684	16	35	2,713 *	172	69	0.40	4,525
Ours $\alpha = 0$	1,170	1,164	4	2	416	303	235	0.78	948 (39%)
Ours $\alpha = 1$	2,283	2,095	8	4	804	282	140	0.50	1,202 (24%)
Ours $\alpha = 2$	4,619	4,166	16	8	1,641	273	84	0.31	1,501 (5%)

¹ All of the related works evaluate their designs on Xilinx Artix-7.

² 36Kb BRAM slices.

³ For works that do not provide #Slices (marked by *), #Slices is approximated by #LUTs $\times$ 0.25 + #FFs $\times$ 0.125 [36].

⁴ #SEC = #BRAMs $\times$ 200 + #DSPs $\times$ 100 + #Slices.

⁵ Number of BRAMs used during the polynomial multiplication.

TABLE III
COMPARISONS OF AREA EFFICIENCY FOR POLYNOMIAL MULTIPLICATION

Work	Freq. (MHz)	Latency (cc)		Time (μs)	ATP (# SEC $\times \mu s$)
		CWM	PMult ¹		
[10]	246	524	2,584	10.50	6,279
[17]	257	536	1,516	5.90	4,909
[35]	159	140	596	3.75	5,381
[18]-1	190	647	2,455	12.92	11,383
[18]-4	182	167	632	3.47	10,184
[18]-16	172	47	187	1.09	12,331
Ours $\alpha = 0$	303	138	608	2.01	2,444 (50%)
Ours $\alpha = 1$	282	88	368	1.30	3,125 (36%)
Ours $\alpha = 2$	273	56	224	0.82	3,970 (20%)

¹ The latency of polynomial multiplication is composed of 1 NTT, 1 CWM, and 1 INTT.

scalable degrees of parallelism, our designs deliver a better tradeoffs between area and time.

The comparisons of polynomial multiplications are given in Table III, which leaves out works that do not provide time performance of a complete polynomial multiplication. The CWM operation spends 138, 88, and 56 cycles including 4, 8, and 11 cycles for reconfiguration. According to (2), the latency of polynomial multiplication is composed of one NTT, one CWM, and one INTT. The ATP in Table III is computed as the product of the #SEC and the run time of polynomial multiplication. It can be seen that our designs all achieve better area efficiency than the previous polynomial multiplication accelerators for Kyber. Especially, our design with $\alpha = 0$ obtains 50% ATP improvement and 1.86 $\times$ speedup to the state-of-the-art work [35]. Compared to the high-speed result of [18], our design with $\alpha = 2$ achieves 1.3 $\times$ speedup and has much better ATP performance.

The proposed polynomial multiplication accelerator can be easily mapped for other algorithms, such as CRYSTALS-Dilithium [23] and Falcon [37]. First, the low-complexity Barrett reduction algorithm (Algorithm 3) can be easily reused

for other NTT primes. For examples $q = 8380417 = 1023 \cdot 2^{13} + 1$ in Dilithium and $12289 = 3 \cdot 2^{12} + 1$ in Falcon, the formula transformation in (9) can be applied to them by setting $(\delta, e) = (1023, 13)$ and $(3, 12)$, respectively. Moreover, the parallel memory access scheme for NTT can be directly employed by them through grouping memories and reusing twiddle factors as mentioned in Section III-C.

V. CONCLUSION

This article presents a reconfigurable and high-efficiency polynomial multiplication accelerator with elaborate optimization methods to facilitate the practical application of Kyber. We modify the Barrett algorithm with a novel formula transformation and propose a low-complexity 24-bit reduction module for Kyber. On the top level, four BUs are combined to form a Bi-Core, which serves as the basic reconfigurable unit. Besides, an efficient parallel memory access scheme is developed utilizing grouping memory and reusing twiddle factors, based on which a compact control logic is designed. Finally, in order to explore different tradeoffs between throughput and area efficiency, our design is parameterized in Verilog and evaluated with 1, 2, and 4 Bi-Cores, respectively. The experiment results deliver better ATP performance than the state-of-the-art works and demonstrate that our scalable design is suitable for different application scenarios with diverse throughput and area requirements. Moreover, the optimizing methods and hardware architecture can be efficiently mapped to other NTT-based algorithms, such as CRYSTALS-Dilithium and Falcon.

REFERENCES

- [1] R. L. Rivest, A. Shamir, and L. M. Adleman, "A method for obtaining digital signatures and public key cryptosystems," in *Secure Communications and Asymmetric Cryptosystems*. London, U.K.: Routledge, 2019, pp. 217–239.
- [2] V. S. Miller, "Use of elliptic curves in cryptography," in *Proc. Conf. Theory Appl. Cryptograph. Techn.*, 1985, pp. 417–426.

- [3] B. Bauer, D. Wecker, A. J. Millis, M. B. Hastings, and M. Troyer, "Hybrid quantum-classical approach to correlated materials," *Phys. Rev. X*, vol. 6, no. 3, 2016, Art. no. 031045.
- [4] P. W. Shor, "Algorithms for quantum computation: Discrete logarithms and factoring," in *Proc. 35th Annu. Symp. Found. Comput. Sci.*, 1994, pp. 124–134.
- [5] D. Moody, "Post-quantum cryptography: NIST's plan for the future," in *Proc. 7th Int. Conf. Post Quantum Cryptography Jpn.*, 2016, pp. 1–9.
- [6] "PQC Standardization Process: Announcing Four Candidates to Be Standardized, Plus Fourth Round Candidates," 2022. [Online]. Available: <https://csrc.nist.gov/News/2022/pqc-candidates-to-be-standardized-and-round-4>
- [7] R. Avanzi et al., "CRYSTALS-Kyber Algorithm Specifications and Supporting Documentation (Version 3.0)," 2020. [Online]. Available: <https://pq-crystals.org/kyber/data/kyber-specification-round3.pdf>
- [8] A. Langlois and D. Stehlé, "Worst-case to average-case reductions for module lattices," *Designs Codes Cryptography*, vol. 75, no. 3, pp. 565–599, 2015.
- [9] Y. Huang, M. Huang, Z. Lei, and J. Wu, "A pure hardware implementation of CRYSTALS-kyber PQC algorithm through resource reuse," *IEICE Electron. Exp.*, vol. 17, no. 17, pp. 1–18, 2020.
- [10] Z. Chen, Y. Ma, T. Chen, J. Lin, and J. Jing, "High-performance area-efficient polynomial ring processor for CRYSTALS-kyber on FPGAs," *Integration*, vol. 78, pp. 25–35, May 2021.
- [11] K. Yao, D.-E.-S. Kundi, C. Wang, M. O'Neill, and W. Liu, "Towards CRYSTALS-kyber: A M-LWE cryptoprocessor with area-time trade-off," in *Proc. IEEE Int. Symp. Circuits Syst. (ISCAS)*, 2021, pp. 1–5.
- [12] Y. Xing and S. Li, "A compact hardware implementation of CCA-secure key exchange mechanism CRYSTALS-kyber on FPGA," in *Proc. IACR Trans. Cryptograph. Hardw. Embedded Syst.*, 2021, pp. 328–356.
- [13] M. Bisheh-Niasar, R. Azarderakhsh, and M. Mozaffari-Kermani, "A monolithic hardware implementation of kyber: Comparing apples to apples in PQC candidates," in *Proc. Int. Conf. Cryptol. Inf. Security Latin America*, 2021, pp. 108–126.
- [14] M. B. Niasar, R. Azarderakhsh, and M. M. Kermani, "Instruction-set accelerated implementation of CRYSTALS-kyber," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 68, no. 11, pp. 4648–4659, Nov. 2021.
- [15] W. Guo, S. Li, and L. Kong, "An efficient implementation of Kyber," *IEEE Trans. Circuits Syst. II, Exp. Briefs*, vol. 69, no. 3, pp. 1562–1566, Mar. 2022.
- [16] M. Bisheh-Niasar, R. Azarderakhsh, and M. Mozaffari-Kermani, "High-speed NTT-based polynomial multiplication accelerator for post-quantum cryptography," in *Proc. IEEE 28th Symp. Comput. Arithmetic (ARITH)*, 2021, pp. 94–101.
- [17] C. Zhang et al., "Towards efficient hardware implementation of NTT for kyber on FPGAs," in *Proc. IEEE Int. Symp. Circuits Syst. (ISCAS)*, 2021, pp. 1–5.
- [18] F. Yarman, A. C. Mert, E. Öztürk, and E. Savaş, "A hardware accelerator for polynomial multiplication operation of CRYSTALS-kyber PQC scheme," in *Proc. IEEE Design Autom. Test Europe Conf. Exhibit. (DATE)*, 2021, pp. 1020–1025.
- [19] P. L. Montgomery, "Modular multiplication without trial division," *Math. Comput.*, vol. 44, no. 170, pp. 519–521, 1985.
- [20] L. Ma, X. Wu, and G. Bai, "Parallel polynomial multiplication optimized scheme for CRYSTALS-kyber post-quantum cryptosystem based on FPGA," in *Proc. IEEE Int. Conf. Commun. Inf. Syst. Comput. Eng. (CISCE)*, 2021, pp. 361–365.
- [21] P. Barrett, "Implementing the Rivest Shamir and Adleman public key encryption algorithm on a standard digital signal processor," in *Proc. Conf. Theory Appl. Cryptograph. Techn.*, 1986, pp. 311–323.
- [22] K. Braun, T. Fritzmann, G. Maringer, T. Schamberger, and J. Sepúlveda, "Secure and compact full NTRU hardware implementation," in *Proc. IFIP/IEEE Int. Conf. Very Large Scale Integr. (VLSI-SoC)*, 2018, pp. 89–94.
- [23] L. Ducas et al., "Crystals-dilithium: A lattice-based digital signature scheme," in *Proc. IACR Trans. Cryptograph. Hardw. Embedded Syst.*, 2018, pp. 238–268.
- [24] S. S. Roy and A. Basso, "High-speed instruction-set coprocessor for lattice-based key encapsulation mechanism: Saber in hardware," in *Proc. IACR Trans. Cryptograph. Hardw. Embedded Syst.*, 2020, pp. 443–466.
- [25] A. C. Mert, E. Öztürk, and E. Savaş, "Design and implementation of a fast and scalable NTT-based polynomial multiplier architecture," in *Proc. IEEE 22nd Euromicro Conf. Digit. Syst. Design (DSD)*, 2019, pp. 253–260.
- [26] T. Pöppelmann and T. Güneysu, "Towards efficient arithmetic for lattice-based cryptography on reconfigurable hardware," in *Proc. Int. Conf. Cryptol. Inf. Security Latin America*, 2012, pp. 139–158.
- [27] V. Lyubashevsky and G. Seiler, "NTTRU: Truly fast NTRU using NTT," Cryptology ePrint Archive, 2019.
- [28] E. Chu and A. George, *Inside the FFT Black Box: Serial and Parallel Fast Fourier Transform Algorithms*. Hoboken, NJ, USA: CRC Press, 1999.
- [29] P. Longa and M. Naehrig, "Speeding up the number theoretic transform for faster ideal lattice-based cryptography," in *Proc. Int. Conf. Cryptol. Netw. Security*, 2016, pp. 124–139.
- [30] W. Liu, S. Fan, A. Khalid, C. Rafferty, and M. O'Neill, "Optimized schoolbook polynomial multiplication for compact lattice-based cryptography on FPGA," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 27, no. 10, pp. 2459–2463, Oct. 2019.
- [31] J. Tian, J. Lin, and Z. Wang, "Fast modular multipliers for super-singular isogeny-based post-quantum cryptography," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 29, no. 2, pp. 359–371, Feb. 2021.
- [32] R. M. Hewlitt and E. Swartzlander, "Canonical signed digit representation for FIR digital filters," in *Proc. IEEE Workshop Signal Process. Syst. SiPS Design Implement.*, 2000, pp. 416–426.
- [33] J. Tian, P. Wang, Z. Liu, J. Lin, Z. Wang, and J. Großschädl, "Efficient software implementation of the SIKE protocol using a new data representation," *IEEE Trans. Comput.*, vol. 71, no. 3, pp. 670–683, Mar. 2022.
- [34] A. C. Mert, E. Karabulut, E. Öztürk, E. Savaş, M. Becchi, and A. Aysu, "A flexible and scalable NTT hardware: Applications from homomorphically encrypted deep learning to post-quantum cryptography," in *Proc. IEEE Design Autom. Test Europe Conf. Exhibit. (DATE)*, 2020, pp. 346–351.
- [35] W. Guo, S. Li, and L. Kong, "An efficient implementation of kyber," *IEEE Trans. Circuits Syst. II, Exp. Briefs*, vol. 69, no. 3, pp. 1562–1566, Mar. 2022.
- [36] "7 Series FPGAs Data Sheet: Overview," 2020. [Online]. Available: https://www.xilinx.com/content/dam/xilinx/support/documents/data_sheets/ds180_7Series_Overview.pdf
- [37] P.-A. Fouque et al., "Falcon: Fast-fourier lattice-based compact signatures over NTRU," in *Proc. NIST*, vol. 36, 2018, p. 5.



Minghao Li received the B.S. degree in integrated circuit design and integrated system from Nanjing University, Nanjing, China, in 2021, where he is currently pursuing the M.S. degree in electronic information.

His research interests include digital signal processing and VLSI design.



Jing Tian (Member, IEEE) received the B.S. degree in microelectronics and the Ph.D. degree in information and communication engineering from Nanjing University, Nanjing, China, in 2015 and 2020, respectively.

She is currently an Associate Researcher with Nanjing University. Her research interests include VLSI design for digital signal processing and cryptographic engineering.



Xiao Hu received the B.S. degree in information engineering from Xi'an Jiaotong University, Xi'an, China, in 2019. She is currently pursuing the Ph.D. degree in information and communication engineering with Nanjing University, Nanjing, China.

Her research interests include VLSI design for cryptographic engineering.



Zhongfeng Wang (Fellow, IEEE) received the B.E. and M.S. degrees from the Department of Automation, Tsinghua University, Beijing, China, in 1988 and 1990, respectively, and the Ph.D. degree from the University of Minnesota, Minneapolis, MN, USA, in 2000.

He has been working with Nanjing University, Nanjing, China, as a Distinguished Professor since 2016. Previously, he worked for Broadcom Corporation, Santa Clara, CA, USA, from 2007 to 2016, as a Leading VLSI Architect. Before that, he worked for Oregon State University, Corvallis, OR, USA, and also with National Semiconductor Corporation, Santa Clara. He has edited one book VLSI and held more than 20 U.S. and China patents. His current research interests are in the area of optimized VLSI design for digital communications and deep learning.

Dr. Wang is a World-Recognized Expert in Low-Power High-Speed VLSI Design for Signal Processing Systems. He has published over 200 technical papers with multiple Best Paper Awards received from the IEEE technical societies, among which is the VLSI Transactions Best Paper Award of 2007. In the current record, he has had many papers ranking among the top 25 most (annually) downloaded manuscripts in IEEE TRANSACTIONS ON VERY LARGE SCALE INTEGRATION (VLSI) SYSTEMS. In the past, he has served as an Associate Editor for IEEE TRANSACTIONS ON CIRCUITS AND SYSTEMS—PART I: REGULAR PAPERS, IEEE TRANSACTIONS ON CIRCUITS AND SYSTEMS—PART II: EXPRESS BRIEFS, and IEEE TRANSACTIONS ON VERY LARGE SCALE INTEGRATION (VLSI) SYSTEMS for many terms. He has also served as a TPC member and various chairs for tens of international conferences. Moreover, he has contributed significantly to the industrial standards. So far, his technical proposals have been adopted by more than 50 international networking standards. In 2015, he was elevated to the Fellow of IEEE for contributions to VLSI design and implementation of FEC coding.